[PUSHREF · AIPUSHTECH · NOTE FONDATRICE]

IA pour développeurs.

Build what's next. Without the noise.

domaine · dev · v1.0

AIPushTech, ce que c'est

L'AIPushTech est une catégorie nouvelle, à l'intersection de trois mondes que personne n'avait encore reliés sérieusement : l'intelligence artificielle, la veille technologique, et l'apprentissage continu.

L'idée tient en une phrase : **l'IA vient à toi, plutôt que l'inverse**. Plutôt que de courir après chaque thread Twitter, chaque newsletter, chaque release de modèle — l'AIPushTech installe un système qui filtre, contextualise, et te confronte à ce qui compte vraiment dans ton domaine. C'est de la veille active sur l'IA appliquée à un métier, doublée d'une logique d'augmentation de compétence : tu ne consommes pas, tu progresses.

Ce que l'AIPushTech n'est pas

- ✗ Pas une plateforme d'*e-learning* avec des modules linéaires et des certifications à afficher sur un CV.
- ✗ Pas un MOOC à consommer une fois pour cocher une case.
- ✗ Pas une école, pas un cours, pas un syllabus figé.

Ce que l'AIPushTech est

- ✓ Une veille structurée par **domaine métier** — l'IA telle qu'elle change ton travail à toi, pas l'IA en général.
- ✓ Un fil d'apprentissage qui suit la sortie réelle des outils, des papiers, des releases.
- ✓ Une discipline intellectuelle : comprendre, pratiquer, retenir.

Pushref, le produit

Ce que Pushref n'est pas

- ✗ Pas un agrégateur RSS qui te déverse mille flux à trier toi-même.
- ✗ Pas un cours en ligne ennuyeux qui t'apprend l'IA en six chapitres et un quiz final.
- ✗ Pas un moteur de recherche où tu dois trouver tes mots-clés à la main, espérer tomber sur la bonne page, et recommencer la semaine suivante.

Ce que Pushref est

- ✓ Une **veille active et passive** sur l'application de l'IA à ton domaine. Active quand tu cherches ; passive quand tu lis simplement le flux du jour.
- ✓ Une **politique éditoriale** propre à chaque domaine — sa devise, son ton, ses critères d'inclusion et d'exclusion.
- ✓ Un système de **sondes** — inclusives, exclusives, boost, strict_exclude — qui scorent chaque article et décident s'il entre dans ton flux ou non.
- ✓ Une sélection rigoureuse de **sources**, pondérées selon leur fiabilité et leur intérêt pour le domaine.

L'ambition est claire : t'aider à **progresser** sur ton métier au rythme de l'IA, sans avoir à devenir documentaliste à temps plein. Tu lis ce qui compte. Tu pratiques en sandbox. Tu retiens via QCM et répétition espacée. Tu partages avec tes pairs.

Comment l'IA a changé le développement logiciel

Marc PERSONA · DOMAINE DEV

32 ans, développeur full-stack senior avec cinq à dix ans d'expérience. TypeScript et Python au quotidien. Utilise Claude Code et Cursor en pair programming permanent. Lit Hacker News et les engineering blogs Anthropic, OpenAI, Vercel. Veut **séparer le signal du bruit** — pas de threads Twitter sensationnalistes, pas de « 5 prompts magiques ». Veut rester en avance d'un cran, durablement, sans s'épuiser à tout suivre.

En cinq ans, le métier de développeur est passé d'un travail d'écriture quasi-artisanale à une pratique **assistée par modèles**, puis à une pratique partiellement **déléguée à des agents**. La vitesse de bascule a été fulgurante. Marc a commencé sa carrière avec Sublime Text et StackOverflow ; écrire du code en 2026 ne ressemble plus du tout à écrire du code en 2021.

Timeline — six jalons qui ont tout déplacé

2021–2022 **GitHub Copilot version 1**
Auto-complétion contextuelle au-delà de la ligne. Le moment où « il y a une IA dans mon IDE » devient banal.

JUIN 2023 **function calling chez OpenAI · RAG mainstream**
L'IA peut appeler des outils. Le pattern Retrieval-Augmented Generation entre en production chez les premières équipes — bases vectorielles, embeddings, contexte injecté.

MARS 2024 **Claude 3 Opus · RAG en production sérieuse**
Les fenêtres de contexte explosent (200k tokens). Les architectures RAG matures s'imposent, les premiers benchmarks production-grade apparaissent.

DÉCEMBRE 2024 **Cursor explose · MCP introduit par Anthropic**
L'IDE où tu parles à ton code devient le standard. Le Model Context Protocol pose les rails d'un écosystème agent-tool standardisé.

2025 **Agents de code autonomes · SWE-bench passe 70 %**
Claude Code en standalone, Codex agentique, Devin. Les modèles dépassent les 70 % sur SWE-bench Verified — ils corrigent des bugs réels dans des repos réels.

2026 **Pair programming agentique généralisé**
Les protocoles A2A (agent-to-agent) émergent. La question légale et culturelle de la propriété du code généré entre dans le débat.

Impacts tangibles aujourd'hui

- **Le coût marginal d'un script change d'ordre de grandeur.** Ce qui prenait deux heures il y a trois ans en prend dix minutes — à condition de savoir prompter et relire.
- **Le pair programming devient asynchrone et asymétrique.** Le « partenaire » n'est plus humain ; il est disponible, inépuisable, et faillible différemment.
- **Le coût d'inférence devient une métrique d'architecture.** Le `token cost` entre dans les revues de code au même titre que la complexité algorithmique.
- **La qualité du prompt devient un nouveau skill rémunéré.** Le *prompt engineering* est sorti de la zone gadget pour entrer dans les fiches de poste.

La mécanique Pushref pour le domaine dev

Pushref ne lit pas le web pour toi. Il lit le web **pour ton domaine**. La différence se joue dans un mécanisme précis : les sondes. Une sonde est un mot-clé chargé d'intention éditoriale ; elle dit « cet article est *pour* ce domaine » ou « cet article *n'est pas* pour ce domaine ». Pushref applique ces sondes à chaque article ingéré, calcule un score, et décide si l'article entre dans le flux du domaine ou non.

> [COMMENT ÇA MARCHE]
>
> Quatre types de sondes interviennent : **inclusive** (poids +1.0, l'article doit en toucher), **boost** (poids +1.5, augmente le score sans être obligatoire), **exclusive** (poids −1.0, pénalise jusqu'à éjection probable), **strict_exclude** (poids −10.0, éjection sans appel). Les sondes alimentent aussi les prompts système d'enrichissement, de QCM et de sandbox — elles cadrent le ton et le périmètre du domaine au-delà du simple filtrage.

Sur `dev`, le système actuel compte **185 sondes** actives en base de production : 127 inclusives, 34 boost, 17 exclusives, 7 strict_exclude. Voici les six sondes les plus emblématiques — celles qui, à elles seules, captent environ 60 % du flux entrant.

Zoom sondes — les six pivots

Claude Code

Capte tout ce qui se passe autour de l'agent de code Anthropic. Releases, intégrations, retours d'expérience production. Sonde à très fort signal — peu de bruit.

Cursor

L'IDE devenu standard du développement assisté. Ses releases, ses comparatifs avec les concurrents (Windsurf, Bolt, Replit), ses cas d'usage industriels.

MCP / Model Context Protocol

Le protocole d'interopérabilité agent-tool introduit par Anthropic. Tout l'écosystème de connecteurs MCP, les implémentations open-source, les debates d'architecture.

agentic / AI agent

Les patterns agent au sens large : multi-step, multi-agent, autonomes. Capte les architectures émergentes et les retours de production.

RAG

Retrieval-Augmented Generation. Toujours dominant en production. Bases vectorielles, embeddings, retrievers hybrides, évaluation RAG.

prompt caching

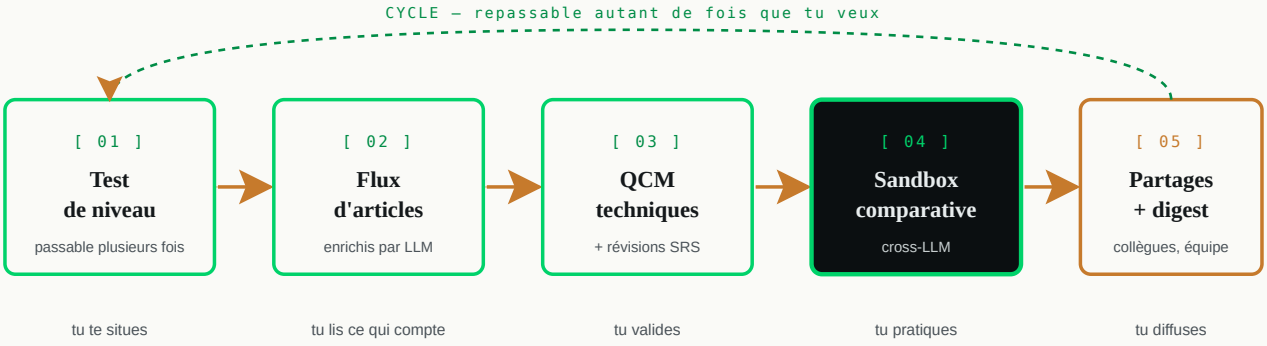
Sonde plus récente, signal d'optimisation production. Capte tout ce qui touche au coût d'inférence, à la latence, aux patterns d'optimisation des appels LLM.

Et inversement — ce que les sondes *excluent*

Les sondes exclusives protègent le flux du bruit qui ferait fuir Marc en moins de cinq minutes : *5 prompts magiques*, *10 hacks AI*, *passive income AI*, *Sam Altman drama*, *AI girlfriend*, *what is ChatGPT*. Tout ce qui appartient à la sphère du contenu d'attention, du clickbait, ou du débutant grand public — tout ça reste dehors. La pureté du flux est un choix éditorial assumé.

Le parcours Pushref, étape par étape

Le parcours Pushref n'est pas linéaire. C'est un **cycle**, et chaque utilisateur peut y entrer à n'importe quel point, en sortir, y revenir. Mais il y a un ordre conseillé qui maximise l'apprentissage.



PARCOURS PUSHREF · DOMAINE DEV

1. Le test de niveau

Vingt-cinq questions qui balayent l'ensemble des sondes du domaine — pas un échantillon, pas un raccourci : la totalité des thématiques que Pushref ingère pour `dev` y est représentée. Trois résultats possibles : **intermédiaire**, **avancé**, **expert**. Le niveau ajuste l'enrichissement des articles, le calibre des QCM, la complexité des prompts proposés en sandbox. Le test est *repassable autant de fois que tu veux* — tu peux changer de niveau quand tu progresses, ou redescendre quand tu attaques un sous-domaine que tu connais moins.

2. Le flux d'articles enrichis

Chaque article du flux a été pré-enrichi par un LLM avec le prompt système du domaine : **contexte**, **résumé**, **points clés**, **questions de réflexion**. Le ton est calibré pour `dev` : technique mais lisible, sans hype, sans réexpliquer ce que tu sais déjà.

3. Le QCM et les révisions SRS

Pour chaque article que tu as ouvert, un QCM auto-généré te propose 4 à 6 questions techniques. Les questions ratées entrent dans une queue de révision en *spaced repetition* à la SuperMemo — tu les revois au bon moment, jamais au mauvais. Pas de gamification agressive, pas de streak quotidien anxiogène : un seul rappel par semaine si la queue dépasse dix items en attente.

4. La sandbox comparative cross-LLM

C'est le cœur de la mécanique de pratique. Tu poses une question — par exemple *« génère-moi un agent Python qui lit un repo GitHub et qui résume les changements depuis le dernier tag »* — et Pushref te montre **côte à côte** les réponses de Claude, GPT, Gemini et Mistral. Tu compares, tu juges, tu retiens. Plusieurs types de sandbox sont prévus à terme (génération de code, revue, débogage, évaluation). Pour l'instant **une seule est disponible — la plus structurante** : la sandbox cross-LLM, relançable autant de fois que tu veux en changeant d'angle (niveau, prompt système custom, paramètres).

5. Les partages

Tu peux partager un article enrichi, un run sandbox, un résultat de QCM, ou ta note hebdo personnelle. Chaque lien est public, sans authentification, frappé de ton pseudo et de ton avatar — avec une OG image dynamique pour LinkedIn, Twitter ou Slack.

Pourquoi ça marche pour le domaine dev

Le domaine `dev` est le **domaine historique de Pushref**. Il n'a pas été imaginé sur tableau blanc puis poussé en bêta : il s'est construit en production, à partir de 3 160 articles ingérés sur plus d'un an, de quinze sources triées sur le volet, et de 185 sondes affinées par itération. C'est lui qui a validé la mécanique du progiciel paramétrable. C'est lui qui sert de référence pour tous les autres domaines en cours d'activation.

Pour Marc — développeur senior, allergique au bruit, passé par Sublime Text, StackOverflow, Copilot v1, puis Claude Code — Pushref offre quelque chose qu'aucun outil ne propose isolément :

- La **discipline éditoriale** de la veille curatée (vs Hacker News brut).
- Le **contexte enrichi** de l'apprentissage assisté (vs lire un article et oublier dans la semaine).
- La **pratique comparée** de la sandbox cross-LLM (vs ouvrir Claude, ouvrir GPT, ouvrir Gemini en trois onglets séparés et perdre la trace).
- L'**ancrage SRS** de la mémorisation à long terme (vs « je l'avais lu il y a six mois mais j'ai oublié »).
- La **capacité de partage** qui transforme un apprentissage individuel en signal social pour l'équipe.

Pas un de ces éléments pris isolément ne suffit. Ce qui fait Pushref, c'est leur combinaison dans un même produit, autour d'un même domaine, avec une politique éditoriale cohérente. C'est cette combinaison que l'AIPushTech revendique comme catégorie.

Trois choses concrètes que tu repartiras avec, après une semaine sur Pushref

- ✓ Un point de vue à jour sur l'agentique de code — pas une intuition, une vraie cartographie d'outils, de patterns, et de limites.
- ✓ Au moins un nouveau pattern d'architecture LLM intégrable dans ta codebase actuelle, validé en sandbox cross-LLM.
- ✓ Une queue de révisions SRS qui t'oblige, sans culpabilité, à revoir au bon moment ce que tu as appris.

> *L'objectif n'est pas que tu deviennes un expert IA en 24 heures. L'objectif est que tu restes en avance d'un cran, durablement, sans t'épuiser à tout suivre.*

■ [06 · CONCLUSION]

Apprendre. Pratiquer. Recommencer.

Pushref ne te promet pas que tu maîtriseras l'IA appliquée au développement en une semaine. Personne ne peut promettre ça honnêtement. Pushref te promet quelque chose de plus utile : **un endroit où tu progresses chaque semaine sans y penser**, en lisant ce qui compte, en pratiquant pour de vrai, en ancrant ce que tu retiens, en partageant avec tes pairs.

C'est de la veille technologique, oui — mais une veille qui te change. C'est de l'apprentissage, oui — mais un apprentissage qui n'a rien d'un cours. C'est un produit, oui — mais un produit qui te rend curieux plutôt qu'épuisé.

Tu rentres quand tu veux. Tu repasses le test de niveau quand tu veux. Tu changes de sandbox d'angle quand tu veux. Tu partages quand tu veux. Tu progresses parce que la mécanique a été pensée pour ça — pas pour te retenir, pas pour te culpabiliser, pas pour te scotcher à un streak quotidien.

— Bienvenue sur Pushref, domaine dev.